

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: Д. Г. Алексеев

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: 05.10.2026

Оценка: _____

Москва, 2026

Условие

Цель работы: приобретение практических навыков управления процессами в ОС и организации обмена данными между процессами с помощью каналов (pipe).

Задание: составить и отладить программу на языке Си, в которой основной процесс создаёт дочерний. Процессы взаимодействуют через каналы. Необходимо обрабатывать системные ошибки.

Вариант №3: Родительский процесс читает имя файла и передаёт его дочернему при создании. Команды вида «число число число» родитель отправляет через pipe1 (связан со стандартным вводом дочернего процесса). Дочерний процесс делит первое число на последующие (тип int, количество чисел произвольное) и пишет результат в файл. Если происходит деление на 0, дочерний и родительский процессы завершаются; проверка деления на 0 выполняется в дочернем процессе. Родитель и дочерний процесс – разные программы; при необходимости дочерний процесс передаёт данные родителю через pipe2.

Метод решения

Программа состоит из двух исполняемых файлов: parent и child. Родитель читает с консоли имя файла, создаёт два канала (pipe1 – от родителя к ребёнку, pipe2 – от ребёнка к родителю) и вызывает fork. В дочернем процессе закрываются неиспользуемые концы каналов, конец pipe1 для чтения переназначается на стандартный ввод (dup2), после чего execl заменяет образ процесса программой child; имя файла и номер дескриптора записи в pipe2 передаются через argv.

Родитель в цикле читает строки пользователя, пишет каждую в pipe1 и блокируется на чтении pipe2: ребёнок на каждую строку отвечает коротким статусом – «OK», «ERR <причина>» либо «DIV0». Такой обмен «запрос – ответ» исключает рассинхронизацию и позволяет родителю сразу показать пользователю ошибку ввода.

Дочерний процесс открывает файл на запись, читает строки из stdin (getline), разбирает их через strtol, делит первое число на все последующие целочисленно и записывает результат в файл. Если встретился ноль, ребёнок пишет об этом в файл, отправляет «DIV0» и завершается с кодом 3; родитель, получив «DIV0», закрывает каналы, ждёт ребёнка (waitpid) и завершается с кодом 2. Если пользователь закончил ввод (Ctrl+D), родитель закрывает pipe1, ребёнок получает EOF и завершается штатно.

Описание программы

Проект состоит из двух программ на Си и файла сборки CMake. В программах нет собственных структур данных: вся работа ведётся с буферами и файловыми дескрипторами.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
parent.c	родительский процесс: читает имя файла, создаёт каналы и дочерний процесс, пересылает строки и обрабатывает ответы
child.c	дочерний процесс: читает строки из stdin, делит числа, пишет результат в файл, отвечает родителю через pipe2

Файл	Назначение
CMakeLists.txt	сборка обеих программ (child собирается вместе с parent)

Основные типы данных и функции

parent.c – main(): весь код родителя (ввод имени файла, pipe/fork/dup2/exec1, цикл обмена, waitpid, код возврата: 0 – штатно, 1 – системная ошибка, 2 – деление на 0).

child.c – static void reply(int fd, const char *msg): отправляет строку-статус родителю через pipe2 (write); main(int argc, char *argv[]): принимает имя файла (argv[1]) и номер дескриптора pipe2 (argv[2]); разбирает числа, проверяет деление на 0 и переполнение int (например, INT_MIN / -1). Коды возврата: 0 – штатно, 1 – ошибка открытия файла или аргументов, 3 – деление на 0.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
int pipe(int fd[2])	создание двух неименованных каналов: pipe1 (родитель → ребёнок) и pipe2 (ребёнок → родитель)
pid_t fork(void)	создание дочернего процесса
int dup2(int old, int new)	переназначение стандартного ввода дочернего процесса на конец чтения pipe1
int execl(const char *path, const char *arg, ...)	замена образа дочернего процесса программой с передачей имени файла и номера дескриптора pipe2
waitpid()	ожидание завершения дочернего процесса и получение его кода возврата
ssize_t read()	чтение ответа ребёнка из pipe2 (родитель) и чтение команд из stdin, то есть из pipe1 (ребёнок)
ssize_t write()	отправка строк в pipe1 (родитель) и запись результатов в файл (ребёнок)
int close(int fd)	заккрытие неиспользуемых концов каналов и закрытие pipe1 родителем даёт ребёнку EOF
_exit(int status)	немедленное завершение дочернего процесса при ошибке dup2 или execl

Ключевые фрагменты кода

Создание каналов и дочернего процесса (parent.c).

Листинг 1 – Создание каналов, fork, dup2 и execl (parent.c)

```
if (pipe(pipe1) == -1) { perror("pipe1"); return 1; }
if (pipe(pipe2) == -1) { perror("pipe2"); return 1; }
pid_t pid = fork();
if (pid == -1) { perror("fork"); return 1; }
if (pid == 0) {
    close(pipe1[1]);
    close(pipe2[0]);
```

```

    if (dup2(pipe1[0], STDIN_FILENO) == -1) { perror("dup2"); _exit(1); }
    close(pipe1[0]);
    char fdStr[16];
    snprintf(fdStr, sizeof(fdStr), "%d", pipe2[1]);
    execl("./child", "child", fileName, fdStr, (char *)NULL);
    perror("execl");
    _exit(1);
}

```

Запись команды в pipe1 и ожидание ответа из pipe2; EPIPE и EOF означают, что ребёнок завершился (parent.c).

Листинг 2 – Обмен «запрос – ответ» с дочерним процессом (parent.c)

```

ssize_t w = write(pipe1[1], line, len);
if (w == -1) { exitCode = 1; break; }
ssize_t r = read(pipe2[0], reply, sizeof(reply) - 1);
if (r == -1) { perror("read pipe2"); exitCode = 1; break; }
if (r == 0) { fprintf(stderr, "Дочерний процесс завершился\n"); exitCode = 1; break; }
reply[r] = '\0';
if (strncmp(reply, "DIV0", 4) == 0) { printf("Деление на 0! Завершаем работу.\n");
exitCode = 2; break; }

```

Проверка деления на 0 на стороне дочернего процесса (child.c).

Листинг 3 – Деление и обработка деления на 0 (child.c)

```

if (count == 0) {
    result = v;
} else {
    if (v == 0) {
        div0 = 1;
        break;
    }
    result /= v;
}
...
if (div0) {
    fprintf(out, "Деление на 0, завершение работы\n");
    fclose(out);
    reply(pipe2fd, "DIV0\n");
    free(line);
    return 3;
}

```

Результаты

Программа собрана через CMake и запущена на Linux (Ubuntu 24.04, gcc, в Docker). Ниже – четыре запуска; строки после приглашений – ввод пользователя (команда запуска: ./parent, файл результатов – result.txt).

Тест 1 – обычные строки, $(84 / 2 / 3 / 7 = 2; -50 / 5 = -10)$; одно число остаётся без изменений):

```

Введите имя файла: result.txt
Вводите строки вида «число число число» (Ctrl+D - выход)
100 5 2
84 2 3 7
-50 5
42
^D
Дочерний процесс завершился с кодом 0
result.txt:

```

```
10
2
-10
42
```

Тест 2 – некорректная строка: родитель получает «ERR» и показывает ошибку, работа продолжается:

```
Введите имя файла: result.txt
Вводите строки вида «число число число» (Ctrl+D - выход)
10 abc 2
Ошибка в строке: некорректная строка
20 4
^D
Дочерний процесс завершился с кодом 0
result.txt:
5
```

Тест 3 – деление на 0: оба процесса завершаются (код ребёнка 3, код родителя 2):

```
Введите имя файла: result.txt
Вводите строки вида «число число число» (Ctrl+D - выход)
100 5 2
7 0 1
Деление на 0! Завершаем работу.
Дочерний процесс завершился с кодом 3
result.txt:
10
Деление на 0, завершение работы
```

Демонстрация работы strace

Ниже – строки strace, относящиеся к заданию, с пояснениями (PID 3614 – родитель, 3615 – дочерний процесс).

Программа запущена командой `strace -f -o strace.txt ./parent`; на вход подано «result.txt», «100 5 2», «7 0 1» (то есть деление на 0 во второй строке). Полный вывод – в приложении Б. А также команда для докера:

```
docker run --rm -it --cap-add SYS_PTRACE --security-opt seccomp=unconfined -v
"$PWD/src":/app osi-lab; gcc -o parent parent.c && gcc -o child child.c
```

```
3614 pipe2([3, 4], 0) = 0
3614 pipe2([5, 6], 0) = 0
3614 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f34628a5a10) = 3615
```

Созданы два канала: `pipe1` – дескрипторы 3 (чтение) и 4 (запись), `pipe2` – 5 (чтение) и 6 (запись). Функция `fork` реализована в `glibc` через `clone`, 3615 – PID ребёнка.

```
3615 close(4 <unfinished ...>)
3615 dup2(3, 0 <unfinished ...>)
3615 execve("./child", ["child", "result.txt", "6"], 0x7ffe54c5dff8 /* 7 vars */) = 0
```

Ребёнок закрывает конец записи `pipe1` (4), переназначает конец чтения (3) на `stdin` (0) и запускает `./child`, передавая имя файла и номер дескриптора 6 (запись в `pipe2`).

```
3615 openat(AT_FDCWD, "result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
3614 write(4, "100 5 2\n", 8 <unfinished ...>)
3615 read(0, "100 5 2\n", 4096) = 8
3615 write(3, "10\n", 3) = 3
3615 write(6, "OK\n", 3 <unfinished ...>)
3614 <... read resumed> "OK\n", 255) = 3
```

Ребёнок открыл файл результата (дескриптор 3), получил команду из pipe1 (read с дескриптора 0), записал в файл результат $100/5/2 = 10$ и ответил «OK» в pipe2, который родитель прочитал.

```
3614 write(4, "7 0 1\n", 6 <unfinished ...>
3615 write(6, "DIV0\n", 5 <unfinished ...>
3614 <... read resumed>"DIV0\n", 255) = 5
3615 exit_group(3 <unfinished ...>
3614 wait4(3615, [{WIFEXITED(s) && WEXITSTATUS(s) == 3}], 0, NULL) = 3615
3614 exit_group(2) = ?
```

Вторая строка содержит ноль: ребёнок отправляет «DIV0» и завершается с кодом 3; родитель через waitpid (wait4) получает этот код и сам завершается с кодом 2.

Выводы

Реализована система из двух программ, взаимодействующих через два неименованных канала: родитель передаёт строки с числами, ребёнок делит первое число на остальные, пишет результат в файл и сообщает родителю статус. Изучены fork, pipe, dup2, exec1, waitpid, а также роль закрытия неиспользуемых концов каналов (без этого ребёнок не получил бы EOF). Корректность подтверждают три теста и вывод strace, в котором видны создание каналов, перенаправление stdin и обмен данными. Для устойчивости игнорируется SIGPIPE, а каждая строка подтверждается ответом ребёнка, что исключает рассинхронизацию. Все системные вызовы проверяются на ошибки.

Приложение А. Исходный код программы

Обязательное приложение: полный код всех файлов проекта (исходные и заголовочные файлы, CMakeLists.txt), каждый файл – отдельным листингом с именем файла в подписи.

Листинг А.1 – Файл parent.c

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <sys/wait.h>

#define BUF_SIZE 4096

int main(void) {
    signal(SIGPIPE, SIG_IGN);

    char fileName[256];
    printf("Введите имя файла: ");
    fflush(stdout);
    if (fgets(fileName, sizeof(fileName), stdin) == NULL) {
        fprintf(stderr, "Не удалось прочесть имя файла\n");
        return 1;
    }
    fileName[strcspn(fileName, "\n")] = '\0';
    if (fileName[0] == '\0') {
        fprintf(stderr, "Имя файла не может быть пустым\n");
        return 1;
    }

    int pipe1[2];
    int pipe2[2];
    if (pipe(pipe1) == -1) { perror("pipe1"); return 1; }
    if (pipe(pipe2) == -1) { perror("pipe2"); return 1; }

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        close(pipe1[1]);
        close(pipe2[0]);

        if (dup2(pipe1[0], STDIN_FILENO) == -1) {
            perror("dup2");
            _exit(1);
        }
        close(pipe1[0]);

        char fdStr[16];
        snprintf(fdStr, sizeof(fdStr), "%d", pipe2[1]);

        execl("./child", "child", fileName, fdStr, (char *)NULL);
        perror("execl");
        _exit(1);
    }
```

```

close(pipe1[0]);
close(pipe2[1]);

printf("Вводите строки вида «число число число» (Ctrl+D - выход)\n");

char line[BUF_SIZE];
int exitCode = 0;

while (fgets(line, sizeof(line), stdin) != NULL) {
    if (line[0] == '\n') continue;
    size_t len = strlen(line);
    if (len == 0) continue;
    if (line[len - 1] != '\n') {
        if (len + 1 < sizeof(line)) { line[len++] = '\n'; line[len] = '\0'; }
    }

    ssize_t w = write(pipe1[1], line, len);
    if (w == -1) {
        if (errno == EPIPE) {
            fprintf(stderr, "Дочерний процесс завершился, выходим\n");
        } else {
            perror("write pipe1");
        }
        exitCode = 1;
        break;
    }

    char reply[256];
    ssize_t r = read(pipe2[0], reply, sizeof(reply) - 1);
    if (r == -1) {
        perror("read pipe2");
        exitCode = 1;
        break;
    }
    if (r == 0) {
        fprintf(stderr, "Дочерний процесс завершился\n");
        exitCode = 1;
        break;
    }
    reply[r] = '\0';

    if (strncmp(reply, "DIV0", 4) == 0) {
        printf("Деление на 0! Завершаем работу.\n");
        exitCode = 2;
        break;
    } else if (strncmp(reply, "ERR", 3) == 0) {
        printf("Ошибка в строке: %s", reply + 4);
    }
}

close(pipe1[1]);
close(pipe2[0]);

int status;
if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}
if (WIFEXITED(status))

```



```

        printf("Дочерний процесс завершился с кодом %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("Дочерний процесс убит сигналом %d\n", WTERMSIG(status));

    return exitCode;
}

```

Листинг A.2 – Файл child.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <limits.h>

static void reply(int fd, const char *msg) {
    if (write(fd, msg, strlen(msg)) == -1)
        perror("child: write pipe2");
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Использование: child <файл> <fd_pipe2>\n");
        return 1;
    }
    const char *fileName = argv[1];
    int pipe2fd = atoi(argv[2]);

    FILE *out = fopen(fileName, "w");
    if (out == NULL) {
        perror("child: fopen");
        reply(pipe2fd, "ERR не удалось открыть файл\n");
        return 1;
    }

    char *line = NULL;
    size_t cap = 0;

    while (getline(&line, &cap, stdin) != -1) {
        char *p = line;
        char *end;
        int count = 0;
        long long result = 0;
        int bad = 0;
        int div0 = 0;

        while (1) {
            while (*p == ' ' || *p == '\t') p++;
            if (*p == '\n' || *p == '\0') break;

            errno = 0;
            long v = strtol(p, &end, 10);
            if (end == p || errno == ERANGE || v > INT_MAX || v < INT_MIN ||
                (*end != ' ' && *end != '\t' && *end != '\n' && *end != '\0')) {
                bad = 1;
                break;
            }
            p = end;
        }
    }
}

```

```

        if (count == 0) {
            result = v;
        } else {
            if (v == 0) {
                div0 = 1;
                break;
            }
            result /= v;
        }
        count++;
    }

    if (div0) {
        fprintf(out, "Деление на 0, завершение работы\n");
        fclose(out);
        reply(pipe2fd, "DIV0\n");
        free(line);
        return 3;
    }
    if (bad || count == 0) {
        reply(pipe2fd, "ERR некорректная строка\n");
        continue;
    }
    if (result > INT_MAX || result < INT_MIN) {
        reply(pipe2fd, "ERR переполнение int\n");
        continue;
    }

    fprintf(out, "%lld\n", result);
    fflush(out);
    reply(pipe2fd, "OK\n");
}

free(line);
fclose(out);
close(pipe2fd);
return 0;
}

```

Листинг А.3 – Файл CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(labaOC2 C)

set(CMAKE_C_STANDARD 11)

add_executable(child child.c)
add_executable(parent parent.c)

# чтобы при запуске parent всегда собирался и child
add_dependencies(parent child)

```

Приложение Б. Полный вывод strace

Обязательное приложение: полный вывод strace без сокращений для запуска из раздела «Демонстрация работы strace». Вывод удобно сохранить в файл: `strace -f -o strace.txt ./parent`. Файл получен запуском в Docker (Ubuntu 24.04) с вводом из pipe: `printf "result.txt\n100 5 2\n7 0 1\n" | strace -f -o strace.txt ./parent`.

Листинг Б.1 – Вывод strace -f ./parent

```
3614 execve("./parent", [ "./parent" ], 0x7ffe985d3068 /* 7 vars */) = 0
3614 brk(NULL) = 0x55ae8c512000
3614 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f34628ab000
3614 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
3614 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
3614 fstat(3, {st_mode=S_IFREG|0644, st_size=10019, ...}) = 0
3614 mmap(NULL, 10019, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f34628a8000
3614 close(3) = 0
3614 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
3614 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"...
, 832) = 832
3614 pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"...
, 784, 64) = 784
3614 fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
3614 pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"...
, 784, 64) = 784
3614 mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f3462600000
3614 mmap(0x7f3462628000, 1609728, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f3462628000
3614 mmap(0x7f34627b1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b1000) = 0x7f34627b1000
3614 mmap(0x7f3462800000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x7f3462800000
3614 mmap(0x7f3462806000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f3462806000
3614 close(3) = 0
3614 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f34628a5000
3614 arch_prctl(ARCH_SET_FS, 0x7f34628a5740) = 0
3614 set_tid_address(0x7f34628a5a10) = 3614
3614 set_robust_list(0x7f34628a5a20, 24) = 0
3614 rseq(0x7f34628a6060, 0x20, 0, 0x53053053) = 0
3614 mprotect(0x7f3462800000, 16384, PROT_READ) = 0
3614 mprotect(0x55ae74817000, 4096, PROT_READ) = 0
3614 mprotect(0x7f34628e9000, 8192, PROT_READ) = 0
3614 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
3614 munmap(0x7f34628a8000, 10019) = 0
3614 rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7f3462645330}, {sa_handler=SIG_DFL,
sa_mask=[], sa_flags=0}, 8) = 0
3614 fstat(1, {st_mode=S_IFCHR|0666, st_rdev=makedev(0x1, 0x3), ...}) = 0
3614 ioctl(1, TCGETS, 0x7ffe54c5c800) = -1 ENOTTY (Inappropriate ioctl for device)
3614 getrandom("\xef\x52\x44\x8f\x32\x18\x78\x79", 8, GRND_NONBLOCK) = 8
3614 brk(NULL) = 0x55ae8c512000
3614 brk(0x55ae8c533000) = 0x55ae8c533000
3614 write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265
\320\270\320\274\321\217 \321\204\320\260\320\271\320\273\320\260"...
, 34) = 34
3614 fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
3614 read(0, "result.txt\n100 5 2\n7 0 1\n", 4096) = 25
3614 pipe2([3, 4], 0) = 0
```

```

3614 pipe2([5, 6], 0) = 0
3614 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f34628a5a10) = 3615
3615 set_robust_list(0x7f34628a5a20, 24 <unfinished ...>
3614 close(3 <unfinished ...>
3615 <... set_robust_list resumed>) = 0
3614 <... close resumed>) = 0
3615 close(4 <unfinished ...>
3614 close(6 <unfinished ...>
3615 <... close resumed>) = 0
3614 <... close resumed>) = 0
3615 close(5 <unfinished ...>
3614 write(4, "100 5 2\n", 8 <unfinished ...>
3615 <... close resumed>) = 0
3614 <... write resumed>) = 8
3615 dup2(3, 0 <unfinished ...>
3614 read(5, <unfinished ...>
3615 <... dup2 resumed>) = 0
3615 close(3) = 0
3615 execve("./child", ["child", "result.txt", "6"], 0x7ffe54c5dff8 /* 7 vars */) = 0
3615 brk(NULL) = 0x559666c0d000
3615 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f9db16a8000
3615 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
3615 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
3615 fstat(3, {st_mode=S_IFREG|0644, st_size=10019, ...}) = 0
3615 mmap(NULL, 10019, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f9db16a5000
3615 close(3) = 0
3615 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
3615 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"...
, 832) = 832
3615 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...
, 784, 64) = 784
3615 fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
3615 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...
, 784, 64) = 784
3615 mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f9db1400000
3615 mmap(0x7f9db1428000, 1609728, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f9db1428000
3615 mmap(0x7f9db15b1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b1000) = 0x7f9db15b1000
3615 mmap(0x7f9db1600000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x7f9db1600000
3615 mmap(0x7f9db1606000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f9db1606000
3615 close(3) = 0
3615 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f9db16a2000
3615 arch_prctl(ARCH_SET_FS, 0x7f9db16a2740) = 0
3615 set_tid_address(0x7f9db16a2a10) = 3615
3615 set_robust_list(0x7f9db16a2a20, 24) = 0
3615 rseq(0x7f9db16a3060, 0x20, 0, 0x53053053) = 0
3615 mprotect(0x7f9db1600000, 16384, PROT_READ) = 0
3615 mprotect(0x55965ae0a000, 4096, PROT_READ) = 0
3615 mprotect(0x7f9db16e6000, 8192, PROT_READ) = 0
3615 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
3615 munmap(0x7f9db16a5000, 10019) = 0
3615 getrandom("\x71\x6a\x46\x09\x5c\xad\xcf\x1a", 8, GRND_NONBLOCK) = 8
3615 brk(NULL) = 0x559666c0d000

```

```

3615 brk(0x559666c2e000) = 0x559666c2e000
3615 openat(AT_FDCWD, "result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
3615 fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
3615 read(0, "100 5 2\n", 4096) = 8
3615 fstat(3, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
3615 write(3, "10\n", 3) = 3
3615 write(6, "OK\n", 3 <unfinished ...>
3614 <... read resumed>"OK\n", 255) = 3
3615 <... write resumed>) = 3
3614 write(4, "7 0 1\n", 6 <unfinished ...>
3615 read(0, <unfinished ...>
3614 <... write resumed>) = 6
3615 <... read resumed>"7 0 1\n", 4096) = 6
3614 read(5, <unfinished ...>
3615 write(3, "\320\224\320\265\320\273\320\265\320\275\320\270\320\265
\320\275\320\260 0, \320\267\320\260\320\262\320\265\321"..., 57) = 57
3615 close(3) = 0
3615 write(6, "DIV0\n", 5 <unfinished ...>
3614 <... read resumed>"DIV0\n", 255) = 5
3615 <... write resumed>) = 5
3614 close(4 <unfinished ...>
3615 exit_group(3 <unfinished ...>
3614 <... close resumed>) = 0
3615 <... exit_group resumed>) = ?
3614 close(5 <unfinished ...>
3615 +++ exited with 3 +++
3614 <... close resumed>) = 0
3614 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3615, si_uid=0,
si_status=3, si_utime=0, si_stime=0} ---
3614 wait4(3615, [{WIFEXITED(s) && WEXITSTATUS(s) == 3}], 0, NULL) = 3615
3614 write(1, "\320\222\320\262\320\276\320\264\320\270\321\202\320\265
\321\201\321\202\321\200\320\276\320\272\320\270 \320\262\320\270"..., 221) = 221
3614 exit_group(2) = ?
3614 +++ exited with 2 +++

```